

Проектирование промпта, MD-«библии» и обёртки журналирования для долгоживущего LLM-агента, способного провести исследование и выпустить дипломную работу

Исполнительное резюме

Цель «долгоживущего исследовательского агента» для диплома — не «генерировать текст», а воспроизводимо управлять исследовательским циклом: искать и отбирать источники, фиксировать решения, поддерживать непротиворечивую память о проекте, писать разделы с проверяемыми ссылками и проходить набор quality gates (проверки качества) на каждом шаге. Для этого обычно недостаточно одной сессии чата; на практике нужны: внешняя память + RAG, инструментальные вызовы (поиск/парсинг/код), оркестрация (граф или контролируемый цикл), наблюдаемость/трассировка и формализованные правила в виде MD-«библии». ¹

На уровне архитектуры наиболее надёжная «базовая конфигурация без ограничений по ресурсам» — оркестратор (state machine/graph) + LLM + инструменты + долговременное хранилище артефактов и индексы (в т.ч. эмбединги) + автоматические проверки (цитирование/фактичность/плагиат/структура) + периодические чекпоинты, пригодные для возобновления работы после прерывания. Это согласуется с линией работ по агентам, сочетающим рассуждения и действия (ReAct), модульной интеграции внешних «экспертов» (MRKL), управлению памятью (MemGPT) и retrieval-augmented подходами (RAG/Self-RAG). ²

Ключевой риск для диплома — «убедительная ложь»: галлюцинации фактов и особенно ссылок (fabricated / hallucinated citations), что эмпирически наблюдается в исследованиях и реальных кейсах; поэтому «верифицируемость» должна быть встроена в дизайн (жёсткие правила источников, хранение доказательств, автоматические проверки DOI/заголовков, запрет на «ссылки из головы»). ³

В отчёте ниже предложены: критерии успеха (качество/глубина/воспроизводимость/верифицируемость/временной бюджет), сравнение архитектур, паттерны промптов, структура MD-«библии», формат прогресс-журналов и чекпоинтов, стратегии контекст-менеджмента, методы оценки и тестирования, план реализации и набор готовых шаблонов.

Цели и критерии успеха агента

Критерии успеха стоит формализовать так, чтобы их можно было проверять автоматически или полуавтоматически (human-in-the-loop), иначе агент неизбежно «оптимизируется на красивый текст». Практически полезно разделить критерии на продуктовые (качество диплома) и процессные (насколько надёжен сам исследовательский конвейер). ⁴

Качество (quality) для диплома удобно разложить на: фактичность и атрибуцию (каждое существенное утверждение поддержано источником), логическую связность и отсутствие внутренних противоречий, корректность терминов и определений, соблюдение структуры научного текста. Подходы retrieval-augmented поколения изначально мотивированы тем, что «параметрические знания» модели ограничены и плохо обновляются, а также тем, что provenance/атрибуция — центральная проблема для генерации. ⁵

Глубина (depth) — это не длина текста, а охват релевантных школ/подходов и качество синтеза. Для агентного дизайна это переводится в требования: (а) систематический поиск литературы, (б) явные критерии включения/исключения, (в) «карта аргументов» и «матрица источников» (какой источник поддерживает какой тезис). В задачах, где требуется стратегический поиск и перебор вариантов, методы расширенного рассуждения (например, дерево мыслей) показывают, что «поиск по пространству решений» может быть принципиально полезен. ⁶

Воспроизводимость (reproducibility) в контексте LLM-агента означает: сохранены входы/выходы, версии промптов и конфигурации, трасса инструментов, а также артефакты (скачанные PDF, фрагменты, извлечённые цитаты). Поскольку модели недетерминированы, простого «повторить запрос» недостаточно; нужны трассировки и журналирование на уровне приложения, как в практиках наблюдаемости LLM-систем. ⁷

Верифицируемость (verifiability) — отдельный критерий: можно ли третьей стороне проверить выводы по логам и источникам. Для диплома это обычно означает: корректные библиографические ссылки и соответствие стандартам оформления (в РФ часто применяется ГОСТ для ссылок и структуры отчёта/НИР как ориентир). ⁸

Временной бюджет (time budget) стоит задавать не только общим лимитом, но и лимитами на итерацию (например: X минут на поиск источников, Y минут на синтез, Z минут на проверку). Иначе агент «залипает» в бесконечный поиск или переписывание, что является типичной проблемой циклических агентных рантаймов; оркестраторы обычно добавляют стоп-условия и лимиты итераций. ⁹

Наконец, из условия задачи: **ограничения по модели/вычислениям/хранилищу не заданы**, поэтому далее рассматриваются подходы, которые могут быть «тяжелее», но надёжнее; при реальном внедрении их можно упрощать, сохраняя контрольные точки и проверки.

Архитектурные опции агента

Таксономия архитектур

Нижне — практическая шкала усложнения (и роста надёжности), которая хорошо согласуется с академическими и промышленными источниками про tool-use, RAG и long-running orchestration. ¹⁰

1) **Одна LLM-сессия (single session)**: максимум — длинный промпт + периодическое ручное «напоминание контекста». Подходит для коротких задач, но ломается на «дипломной длине» из-за ограничений окна контекста и деградации использования длинного контекста (позиционный эффект: «лучше помнит начало и конец», хуже — «середину»). ¹¹

2) **Цепочка сессий (chained sessions)**: отдельные сессии на этапы (поиск → конспект → план → черновик → редактирование), где каждый шаг производит артефакт-резюме для следующего. Это

уже близко к «оркестрации», но без инструментов сложно обеспечить проверяемость, а без памяти качество деградирует при длительных проектах. ¹²

3) **Внешняя память + RAG (external memory / RAG)**: источники и собственные артефакты диплома (заметки, конспекты, фрагменты) индексируются; в контекст подаются только релевантные куски. Классическая мотивация RAG — снизить зависимость от «параметрической памяти» и улучшить фактичность/обновляемость знаний. ⁵

4) **Агент с инструментами (tool-using agent)**: модель решает, когда вызвать поиск, парсер PDF, выполнение кода и т.п. Паттерн ReAct формализует чередование «мысленного шага» и «действия», показывая пользу от связки рассуждения и обращения к внешним источникам для снижения галлюцинаций в задачах QA/верификации. ¹³

5) **Модульная/оркестрированная система (MRKL-стиль, граф/машина состояний)**: LLM — не «всё», а контроллер или компонент; рядом специализированные модули (поиск, базы знаний, компилятор, валидаторы, менеджер памяти). Это повышает надёжность и управляемость, но увеличивает сложность. ¹⁴

Базовая референс-архитектура для дипломного агента

```
flowchart TB
    U[Пользователь / Научный руководитель] -->|требования, замечания| ORCH
    subgraph ORCH [Оркестратор: цикл/граф]
        ORCH --> LLM[LLM: планирование, синтез, генерация]
        ORCH --> QA[Quality Gates: проверки]
        LLM -->|tool call| TOOLS[Инструменты: поиск, PDF, код, цитаты]
        TOOLS --> WEB[Поиск/веб]
        TOOLS --> PDF[PDF/парсинг источников]
        TOOLS --> CODE[Выполнение кода/эксперименты]
        TOOLS --> CITE[Менеджер ссылок/проверка DOI]
        ORCH --> MEM[Память: заметки, чанки, векторный индекс]
        MEM --> LLM
        ORCH --> LOG[Журналы: trace, прогресс, чекпоинты]
        QA -->|pass/fail| ORCH
    end
```

Такой дизайн отражает: (а) необходимость инструментов и модулей (MRKL), (б) агентное чередование «думать/действовать» (ReAct), (в) долговременную память с управлением «уровнями» (MemGPT), (г) retrieval-augmented синтез и атрибуцию. ¹⁵

Сравнительная таблица альтернатив

Оценки ниже **качественные** (порядковые), потому что реальные задержки/стоимость сильно зависят от модели, частоты инструментов и объёма документов.

Архитектура	Задержка (latency)	Стоимость (cost)	Надёжность (reliability)	Сложность (complexity)	Где подходит лучше всего
Одна сессия	низкая	низкая	низкая	низкая	короткие обзоры, наброски ¹⁶

Архитектура	Задержка (latency)	Стоимость (cost)	Надёжность (reliability)	Сложность (complexity)	Где подходит лучше всего
Цепочка сессий + резюме	средняя	средняя	средняя	средняя	диплом «вручную-с-помощью», без строгой верификации ¹⁷
Внешняя память + RAG	средняя	средняя	высокая	средняя	систематический лит-обзор и атрибутируемые утверждения ¹⁸
Агент с инструментами (ReAct-подобно)	высокая	высокая	высокая	высокая	многодневное исследование с поиском/парсингом/кодом ¹⁹
Оркестратор-граф + quality gates	высокая	высокая	очень высокая	очень высокая	«надёжный производственный конвейер» диплома, resume-после-сбоев ²⁰

Если выбирать «по умолчанию» для дипломного проекта, то минимально жизнеспособная надёжная конфигурация: **RAG + строгие правила цитирования + чекпоинты**, а при наличии инженерных ресурсов — **оркестратор-граф + мониторинг/трассировка**. ²¹

Паттерны промпт-дизайна

Роли и приоритеты инструкций

Практика разделения инструкций по уровням («системные правила» vs «задача пользователя») полезна для устойчивости: системный слой фиксирует неизменяемые требования (верификация, стиль, запреты), а пользовательский — текущую задачу итерации. В промышленных руководствах по системным промптам подчёркивается именно роль system-уровня как задания роли/поведения, а не места для всего подряд. ²²

Отдельно важно учитывать, что API-контуры могут меняться (например, у OpenAI ²³ объявлялись депрекации и миграции инструментальных API), поэтому промпт-библию и оркестратор стоит проектировать «провайдер-агностично» и хранить как файлы/версии в репозитории. ²⁴

Декомпозиция и «управляемое рассуждение»

Для длинных задач (диплом) работает правило: «сначала структура и подзадачи, затем решение». Это формализовано в паттернах least-to-most (от декомпозиции к решению) и в более общем виде — search-подходах типа tree-of-thoughts, где модель исследует несколько ветвей и отбирает лучшие. ²⁵

Chain-of-thought (генерация промежуточных шагов) эмпирически улучшает решение многосоставных задач; self-consistency показывает, что выбор ответа по согласованности нескольких сэмплов может повышать точность. ²⁶

При этом для «надёжного дипломного конвейера» важно различать: - **скрытое рассуждение** (модель может «думать», но вы не обязаны сохранять весь трасс в диплом); - **видимые обоснования** (короткие проверяемые пояснения + ссылки на источники).

Такой компромисс сохраняет пользу «пошаговости» для качества, но делает результат проверяемым и более безопасным для публикации/передачи руководителю. ²⁷

Инструментальные вызовы и структурированные выходы

ReAct-подход полезен как «промтовая единица» для агента: модель чередует рассуждение и действие (поиск/извлечение фактов), что снижает риск галлюцинаций на задачах, требующих внешних знаний. ¹³

Для вызовов инструментов (function/tool calling) важно **жёстко ограничить формат**: JSON-схемы, валидаторы аргументов, явное разделение «tool call» и «tool output». Похожие принципы отражены в документации инструментальных API и во фреймворках агентной интеграции. ²⁸

Мини-шаблон «инструкции итерации»

Ниже — не «магическая формула», а структурный шаблон, который снижает вероятность расползания задач и делает логи пригодными для проверки.

```
# Итерация N – Задача
## Контекст
- Кратко: что уже сделано (ссылка на чекпоинт/лог)
- Ограничения: стиль, формат, язык, объём

## Цель итерации (1–2 предложения)
- ...

## Входы (артефакты)
- notes/...
- sources/...
- outline/...

## План (до 7 шагов)
1) ...
2) ...

## Требования к доказательности
- Для каждого ключевого утверждения: (а) источник, (б) цитируемый фрагмент/
  страницы, (в) уровень уверенности.

## Выходы
- deliverable_1.md
- update_checkpoint.md
```

```
- log_entry.md

## Стоп-условия
- максимум К инструментальных запросов
- максимум Т минут / токенов
```

Принцип «контекст-инжиниринга» (организация контекста как инженерная дисциплина, а не только «красивый промпт») отдельно подчёркивается в современных практических руководствах. ²⁹

Структура MD-«библии» (проектного устава агента)

MD-«библия» — это стабильный, версионированный документ (или набор документов), который определяет «конституцию» агента: стандарты качества, правила использования источников, формат артефактов, а также **политику сбоев** (что делать, если источников нет, если найден конфликт, если инструмент упал). Это особенно важно из-за документированных проблем с вымышленными ссылками и ошибками атрибуции в LLM-генерации. ³⁰

Рекомендуемые разделы MD-«библии»

Ниже — структура, закрывающая требования из запроса (decision rules, failure modes, escalation policies) и стыкуемая с ГОСТ-ориентированным академическим оформлением.

- 1) **Миссия и рамки:** какой диплом, какой предмет, какие «не делаем».
- 2) **Цели и метрики:** критерии успеха (качество/глубина/воспроизводимость/верифицируемость/время).
- 3) **Политика источников:** приоритет первичных/официальных и недопустимость «ссылок из головы».
- 4) **Политика цитирования и оформление:** требования к библиографическим ссылкам; в РФ часто применяют ГОСТ на библиографическую ссылку и стандартные требования к структуре отчёта/НИР как ориентир для структуры и формальных элементов. ⁸
- 5) **Решения и правила** (decision rules): когда выполнять поиск, когда достаточно памяти, сколько источников нужно на обзор, как разрешать конфликтующие источники.
- 6) **Сценарии отказа** (failure modes): галлюцинации фактов/ссылок, «потеря середины контекста», заикание агента, prompt injection, деградация качества при длинном контексте. ³¹
- 7) **Эскалация** (escalation): когда требуется человек (руководитель/автор) — например, спорные утверждения, этические вопросы, невозможность верификации.
- 8) **Безопасность и этика:** запрет на утечки персональных данных, политика обработки конфиденциальных материалов, защита от prompt injection. ³²
- 9) **Требования к прозрачности использования AI:** как фиксировать вклад инструмента и кто несёт ответственность за итоговый текст (в духе рекомендаций по ответственности/раскрытию).

³³

Пример «ядра» MD-библии

```
# MD-Библия дипломного агента (vX.Y)

## Принципы
- Истинность важнее беглости.
```

- Ни одного библиографического источника "из головы".

-

Каждый ключевой тезис должен иметь трассируемую опору: (источник → фрагмент → тезис).

Политика источников

- Приоритет: (1) стандарты/официальная документация, (2) рецензируемые статьи, (3) препринты (с пометкой), (4) вторичные обзоры.
- Любая ссылка должна быть проверена: заголовок, авторы/организация, год, идентификатор (DOI/ISBN/URL), доступность.

Политика цитирования

-

Формат ссылок: согласно требованиям ВУЗа; если не задано – ориентируемся на ГОСТ Р 7.0.5-2008.

- Для каждого раздела диплома: минимум N ссылок на первичные источники.

Decision rules

- Если утверждение относится к фактам/цифрам/датам: обязателен поиск + минимум 2 независимых источника.
- Если в источниках конфликт: фиксируем конфликт в журнале, приводим обе позиции, не сглаживаем.

Failure modes

- Галлюцинация ссылок → немедленный стоп, запуск "citation audit".
- Потеря контекста → создание чекпоинта и перезапуск с retrieval по памяти.

-

Зацикливание → лимит итераций, переключение стратегии (decompose → retrieve → write).

Эскалация

- Неверифицируемые ключевые тезисы, этика/право, спорные выводы → запрос решения у автора/руководителя.

Этический и редакционный контур важен: крупные издатели и комитеты по публикационной этике прямо требуют прозрачности использования AI и подчёркивают, что ответственность сохраняется за человеком. ³³

Форматы прогресс-журналов и стратегии управления контекстом

Прогресс-журналы: зачем они нужны и что фиксировать

Дипломный агент должен быть «наблюдаемым»: любая итерация оставляет след, чтобы можно было (а) воспроизвести ход работы, (б) найти источник ошибки, (в) восстановиться после сбоя. В индустриальной терминологии это близко к трассировкам/observability, где сохраняется полный trace запроса и вызовов компонентов. ³⁴

Минимально полезные сущности логирования:

- **Trace-лог** (подробный): входы/выходы, tool calls, статусы валидаторов, метрики (токены/ время/стоимость).
- **Progress-журнал** (компактный): что сделано, какие решения приняты, что дальше, какие риски.
- **Чекпоинт** (редко, но тщательно): консистентное состояние проекта, которое можно подать в контекст для resume.

Рекомендуемый формат записи (компактно, но структурно)

```
# Progress Log Entry
- run_id: 2026-02-22T18-30Z__thesis
- iter: 47
- stage: literature_review / outline / writing / verification
- objective: "Собрать первичные источники по RAG evaluation и выписать метрики"
- inputs:
  - memory_refs: [mem://notes/rag_eval, mem://bib/core]
  - new_queries: ["RAGAs faithfulness metric", "LLM-as-a-judge bias MT-Bench"]
- outputs:
  - artifacts: [notes/rag_eval.md, bib/rag_eval.bib]
  - drafted_text: sections/2_evaluation.md
- decisions:
  - "Вводим метрики RAGAs: context precision/recall, faithfulness, answer relevancy"
- verification:
  - citation_audit: PASS (12/12)
  - factuality_check: WARN (2 claims need second source)
- next:
  - "Собрать источники по hallucinated citations и оформить policy"
- risks:
  - "Смещение LLM-judge в сторону verbose ответов"
```

Для больших проектов полезна **ротация/архивация**: - «горячая зона» (hot set): последние N итераций + текущий план + открытые вопросы; - «тёплая зона»: недельные чекпоинты и «карта аргументов»; - «холодная зона»: полный trace-архив (сжатый) + артефакты.

Стандартизировать метаданные можно в духе общих практик телеметрии: ресурс/контекст + корреляция логов/трейсов. ³⁵

Эмбединги и поиск по журналам

Чтобы агент реально использовал журналы, их нужно не только хранить, но и **делать извлекаемыми**: сохранять краткие «summary-чанки» каждой итерации и индексировать их для retrieval. Подобные паттерны (поток событий → рефлексия → retrieval при планировании) показаны и в архитектурах генеративных агентов, и в системах управления памятью. ¹²

Контекст-менеджмент: стратегии и их компромиссы

Проблема длинного диплома — не только ограничение окна контекста, но и качество чтения длинного контекста: модели часто хуже извлекают релевантное «из середины» длинного промпта. ¹¹

Практический набор стратегий:

- **Chunking + retrieval (RAG)**: разбиение источников/проектных заметок на чанки, индекс, топ-К извлечение в контекст. ¹⁸
- **Суммаризация и многоуровневая память**: периодически «упаковывать» историю в краткие чекпоинты (аналог иерархий памяти). ³⁶
- **Компрессия промпта**: методы типа LLMlingua предлагают алгоритмическую компрессию промптов для экономии токенов/ускорения при сохранении смысла, что полезно при длинных проектах. ³⁷
- **Окно + приоритизация + вытеснение**: явно хранить приоритеты «что обязательно должно быть в контексте», а остальное извлекать по запросу. Этот принцип близок к «виртуальному контекст-менеджменту». ³⁶

Сравнение (качественно):

Стратегия	Latency	Cost	Надёжность	Сложность	Комментарий
Длинный промпт без retrieval	низк./средн.	высокая	низкая	низкая	страдает от «lost in the middle» ¹⁶
Chunking + RAG	средняя	средняя	высокая	средняя	базовый выбор для диплома ³⁸
Иерархическая память (чекпоинты)	средняя	средняя	высокая	средняя	хорошо для resume-после-сбоев ³⁶
Компрессия промптов	низкая/средняя	низкая/средняя	средняя	высокая	оптимизация после стабилизации пайплайна ³⁹

Диаграмма «петли» агента с журналированием и чекпоинтами

```
flowchart TD
    A[Загрузить чекпоинт + задачи] --> B[Сформировать контекст: policies + retrieval]
    B --> C{Нужны внешние данные?}
    C -- да --> D[Tool call: поиск/PDF/код]
    D --> E[Занести результаты в память + источники]
    C -- нет --> F[Синтез/написание фрагмента]
    E --> F
    F --> G[Quality gates: цитаты, факты, формат]
    G -- fail --> H[Локальная корректировка / переформулировка / доп.источник]
    H --> B
    G -- pass --> I[Запись Progress log]
```

```
I --> J{Чекпоинт пора?}
J -- да --> K[Сформировать checkpoint summary + backlog]
J -- нет --> L[Следующая итерация]
K --> L
```

Оценка качества, тестирование и контроль академической добросовестности

Автоматические и полуавтоматические проверки (quality gates)

Для дипломного агента полезно иметь «конвейер» проверок, часть из которых можно автоматизировать:

- **Проверка цитат (citation audit):** существуют исследования, показывающие, что модели часто изобретают библиографические ссылки; зафиксированы и большие доли «fabricated references» в экспериментах и наблюдениях. Поэтому правило: «нет проверенного источника — нет ссылки в тексте» должно быть автоматизировано. ³⁰
- **Проверка фактичности/галлюцинаций:** есть методы для детекции галлюцинаций в «чёрном ящике», например sampling-подходы, и более общие статистические оценки неопределённости, опубликованные в рецензируемых работах. ⁴⁰
- **Проверка «faithfulness» для RAG:** метрики RAGAs нацелены на оценку связи retrieval+generation (precision/recall контекста, faithfulness, relevancy) без необходимости полного набора разметки. ⁴¹
- **Проверки безопасности:** prompt injection и связанные риски для инструментальных агентов выделяются как ключевые уязвимости; это важно, если агент читает внешние страницы или документы и затем выполняет действия. ³²

Протоколы оценки: unit-тесты, регрессии, бенчмарки, human-in-the-loop

Для инженерной части агента (парсинг PDF, извлечение метаданных, форматирование библиографии, сбор артефактов) применимы обычные unit/integration тесты. Для «интеллектуальной» части нужны регрессии на фиксированном наборе задач: одинаковый запрос → сравнить качество по метрикам и чеклисту. Это особенно важно из-за недетерминированности и дрейфа моделей; инструменты наблюдаемости прямо мотивируют необходимость трасс и мониторинга для LLM-приложений. ⁴²

Для автоматизированной оценки «текста как текста» часто применяют LLM-оценщиков (LLM-as-a-judge): MT-Bench/Chatbot Arena исследуют и ограничения такого подхода (например, смещения по многословности) и дают практические ориентиры. G-Eval показывает, как LLM-оценка может коррелировать с человеческими оценками для ряда NLG-задач, но также требует осторожности из-за возможных bias. ⁴³

Для инфраструктуры eval полезны готовые модули оценки в RAG-фреймворках и библиотеках оценки (например, документация по evaluation в LlamaIndex ⁴⁴ и библиотеки общего назначения). ⁴⁵

Контур академической добросовестности и раскрытия использования AI

Отдельный слой требований — ответственность автора и прозрачность. Позиции COPE ⁴⁶ и ICMJE ⁴⁷ подчёркивают, что AI-инструменты не могут быть авторами и что использование AI

должно корректно отражаться/раскрываться (в зависимости от политики площадки/ВУЗа). Политики крупных издателей (например, Elsevier ⁴⁸) также требуют oversight и disclosure и ограничивают использование генеративных инструментов в некоторых этапах редакционного процесса. ³³

Пример тестового чеклиста (минимум для дипломной итерации)

- Источники: все ключевые утверждения имеют проверяемые ссылки; нет «HalluCitation-паттернов» (несуществующие журналы/тома/страницы). ⁴⁹
- Доказательность: для спорных/количественных утверждений — минимум 2 независимых источника или пометка о неопределённости. ³⁸
- Целостность: в разделе нет внутренних противоречий; термины определены. ⁵⁰
- RAG-качество (если применимо): faithfulness/relevancy по выбранной методике (например, RAGAs). ⁵¹
- Безопасность: внешние документы не «инструктируют» агента менять политику; промпт-инъекции не приводят к действиям. ³²
- Логи: итерация имеет progress entry + trace; артефакты сохранены; чекпоинт обновлён по расписанию. ⁵²

План внедрения и набор шаблонов

Этапы реализации (инженерно-исследовательский план)

Эволюционно надёжнее строить систему от «верифицируемого минимума» к «умной оркестрации», иначе получится сложный агент без контроля качества.

- 1) **Версионизируемая MD-библия + структура проекта** (репозиторий): правила, форматы, папки артефактов, соглашения именования, baseline-чеклист. ⁵³
- 2) **Журналы и артефакты**: прогресс-лог + trace-лог + чекпоинт; хранение в файловой и/или трекинг-системе (идеологически похоже на tracking/artefacts подходы в системах экспериментов). ⁵⁴
- 3) **RAG-слой**: индекс источников и заметок; строгая политика «в контекст только retrieved-чанки + ссылки». ⁵⁵
- 4) **Инструменты**: поиск, загрузка PDF, извлечение метаданных/цитат; строгие схемы tool calls. ⁵⁶
- 5) **Quality gates**: citation audit, фактичность/галлюцинации, структурные проверки; подключить RAGAs и/или LLM-judge для регрессий. ⁵⁷
- 6) **Оркестрация графом + human-in-the-loop**: формализовать переходы стадий, лимиты, эскалации. Фреймворки оркестрации подчёркивают ценность stateful long-running агентов и встроенных human-in-the-loop механизмов. ⁵⁸
- 7) **Оптимизация контекста**: компрессия и приоритизация, если стоимость/латентность становятся проблемой. ³⁷

Шаблон инициализации (system)

SYSTEM: Ты — исследователь и исполнитель дипломного проекта.

Неподвижные правила:

- Никогда не придумывай источники и библиографические записи.
- Любое существенное фактическое утверждение должно быть поддержано проверяемым источником или помечено как предположение.

- Работай итеративно: план → действия (поиск/извлечение) → синтез → проверки → журнал → чекпоинт.
- Соблюдай формат артефактов проекта и правила MD-библии.

Если проверка провалена (цитаты/факты/структура) – сначала исправляй, затем продолжай.

Выход: только запрошенный артефакт + краткий прогресс-лог.

Этот шаблон намеренно «скучный»: лучшие практики промптинга предупреждают против избыточной сложности и «over-engineering» в инструкциях — лучше добавлять правила по мере выявления конкретных сбоев. ⁵⁹

Шаблон итеративного цикла (user)

USER: Итерация {iter}

Стадия: {stage}

Цель: {objective}

Доступные артефакты:

- checkpoint: checkpoints/{last}.md
- оглавление: outline.md
- заметки: notes/
- источники: bib/ и sources/

Задача:

- 1) Сформируй план на ≤ 7 шагов.
- 2) Если нужны внешние данные – иницируй tool calls.
- 3) Напиши/обнови {deliverable}.
- 4) Прогони проверки: citation_audit + consistency.
- 5) Заполни progress log entry.

Ограничения:

- максимум {K} tool calls
- итоговый текст раздела $\leq \{N\}$ слов

Шаблон «resume после прерывания»

USER: Мы возобновляем проект.

Вот последний чекпоинт и хвост журнала: {paste_checkpoint + last_log_entries}

Сделай:

- Сверх текущее состояние (что готово, что не готово).
- Восстанови backlog задач (приоритеты).
- Выбери одну следующую итерацию с самым высоким ROI (качество/верификация/сроки).
- Сгенерируй итерационный план и начни выполнение.

Чекпоинт-ориентированный «resume» согласуется с идеей многоуровневой памяти и управляемого перемещения информации в «рабочий контекст». ⁶⁰

Компромиссы, риски и меры снижения

Карта рисков

1) **Галлюцинации фактов и ссылок**: исследования прямо фиксируют проблему fabricated bibliographic citations; для дипломного агента это «красная зона» риска. Митигировать: запрет на ссылки без проверки, автоматический citation audit, хранение доказательств (скрин/страницы), retrieval-only цитирование. ⁶¹

2) **Деградация в длинном контексте** («lost in the middle») и дрейф фокуса: митигировать через RAG, «горячий набор» приоритетной информации, чекпоинты, компрессию. ⁶²

3) **Prompt injection и небезопасное поведение инструментального агента**: особенно если агент читает веб-страницы и затем выполняет действия. Митигировать: изоляция инструментов, явные allow-lists, «невыполнимость инструкций из внешних документов», тесты по OWASP. ³²

4) **Смещения автоматической оценки (LLM-judge bias)**: LLM-оценщики масштабируемы, но не нейтральны (например, к многословию). Митигировать: комбинировать LLM-judge с человеческим ревью, фиксированными критериями и задачами-эталоном, а также RAG-метриками типа RAGAs там, где это уместно. ⁶³

5) **Этические/академические требования**: риск несоответствия политикам ВУЗа/издателей. Митигировать: раздел «disclosure» в дипломе, фиксация роли AI, ответственность автора, соответствие требованиям оформления ссылок/структуры. ⁶⁴

6) **Технологический дрейф API/моделей**: инструментальные интерфейсы деприкейтятся; без абстракции и логов воспроизводимость ломается. Митигировать: провайдер-агностичный слой, контрактные тесты tool calls, сохранение версий и deprecation-мониторинг. ⁶⁵

Иллюстративная «карта компромиссов» (надёжность vs сложность)

```
quadrantChart
  title Надёжность vs Сложность (качественная оценка)
  x-axis низкая сложность --> высокая сложность
  y-axis низкая надёжность --> высокая надёжность
  quadrant-1 "сложно, но надёжно"
  quadrant-2 "просто и надёжно"
  quadrant-3 "просто, но ненадёжно"
  quadrant-4 "сложно и ненадёжно"
  "Одна сессия": [0.15, 0.25]
  "Цепочка сессий": [0.35, 0.45]
  "RAG+память": [0.55, 0.70]
  "Tool-агент": [0.75, 0.78]
  "Граф+gates+HITL": [0.90, 0.90]
```

Главная практическая рекомендация из этой карты: если вы не готовы строить «графовую оркестрацию», то всё равно стоит внедрить **ядро надёжности** — RAG-память, правила верификации, журналирование и чекпоинты; именно они закрывают самые опасные риски (ложные факты/ссылки и невоспроизводимость). ⁶⁶

¹ ⁵ ¹⁸ ²¹ ³⁸ ⁴⁸ ⁵⁵ ⁶⁶ Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks
https://arxiv.org/abs/2005.11401?utm_source=chatgpt.com

² ¹⁰ ¹³ ¹⁹ ⁵⁶ ReAct: Synergizing Reasoning and Acting in Language Models
https://arxiv.org/abs/2210.03629?utm_source=chatgpt.com

³ ³⁰ ⁴⁹ ⁶¹ Fabrication and errors in the bibliographic citations ...
https://www.nature.com/articles/s41598-023-41032-5?utm_source=chatgpt.com

⁴ ⁷ ³⁴ ⁴² ⁵² Tracing quickstart - Docs by LangChain
https://docs.langchain.com/langsmith/observability-quickstart?utm_source=chatgpt.com

⁶ Tree of Thoughts: Deliberate Problem Solving with Large Language Models
https://arxiv.org/abs/2305.10601?utm_source=chatgpt.com

⁸ ГОСТ Р 7.0.5-2008 Библиографическая ссылка. Общие ...
https://www.ifap.ru/library/gost/7052008.pdf?utm_source=chatgpt.com

⁹ Agents - Docs by LangChain
https://docs.langchain.com/oss/python/langchain/agents?utm_source=chatgpt.com

¹¹ ¹⁶ ³¹ ⁴⁶ ⁶² Lost in the Middle: How Language Models Use Long Contexts
https://arxiv.org/abs/2307.03172?utm_source=chatgpt.com

¹² ¹⁷ Generative Agents: Interactive Simulacra of Human Behavior
https://arxiv.org/abs/2304.03442?utm_source=chatgpt.com

¹⁴ ¹⁵ MRKL Systems: A modular, neuro-symbolic architecture that combines large language models, external knowledge sources and discrete reasoning
https://arxiv.org/abs/2205.00445?utm_source=chatgpt.com

²⁰ ²³ ⁴⁴ ⁵⁸ LangGraph overview - Docs by LangChain
https://docs.langchain.com/oss/python/langgraph/overview?utm_source=chatgpt.com

²² Giving Claude a role with a system prompt - Claude API Docs
https://docs.anthropic.com/en/docs/build-with-claude/prompt-engineering/system-prompts?utm_source=chatgpt.com

²⁴ ⁶⁵ Deprecations | OpenAI API
https://developers.openai.com/api/docs/deprecations/?utm_source=chatgpt.com

²⁵ ⁵⁰ LEAST-TO-MOST PROMPTING ENABLES COMPLEX ...
https://webdocs.cs.ualberta.ca/~dale/papers/iclr23a.pdf?utm_source=chatgpt.com

²⁶ ²⁷ Chain-of-Thought Prompting Elicits Reasoning in Large Language Models
https://arxiv.org/abs/2201.11903?utm_source=chatgpt.com

²⁸ Assistants Function Calling | OpenAI API
https://developers.openai.com/api/docs/assistants/tools/function-calling/?utm_source=chatgpt.com

²⁹ ⁵³ Effective context engineering for AI agents
https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents?utm_source=chatgpt.com

³² OWASP Top 10 for Large Language Model Applications
https://owasp.org/www-project-top-10-for-large-language-model-applications/?utm_source=chatgpt.com

33 64 **Authorship and AI tools | COPE**

https://publicationethics.org/guidance/cope-position/authorship-and-ai-tools?utm_source=chatgpt.com

35 **OpenTelemetry Logging**

https://opentelemetry.io/docs/specs/otel/logs/?utm_source=chatgpt.com

36 60 **MemGPT: Towards LLMs as Operating Systems**

https://arxiv.org/abs/2310.08560?utm_source=chatgpt.com

37 39 **LLMLingua: Compressing Prompts for Accelerated Inference of Large Language Models**

https://arxiv.org/abs/2310.05736?utm_source=chatgpt.com

40 **SelfCheckGPT: Zero-Resource Black-Box Hallucination Detection for Generative Large Language Models**

https://arxiv.org/abs/2303.08896?utm_source=chatgpt.com

41 47 51 57 **Automated Evaluation of Retrieval Augmented Generation**

https://arxiv.org/abs/2309.15217?utm_source=chatgpt.com

43 63 **Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena**

https://arxiv.org/abs/2306.05685?utm_source=chatgpt.com

45 **Evaluating | LlamaIndex Python Documentation**

https://developers.llamaindex.ai/python/framework/module_guides/evaluating/?utm_source=chatgpt.com

54 **MLflow Tracking**

https://mlflow.org/docs/latest/ml/tracking/?utm_source=chatgpt.com

59 **Prompting best practices - Claude API Docs**

https://docs.anthropic.com/en/docs/build-with-claude/prompt-engineering/claude-4-best-practices?utm_source=chatgpt.com